CS 450/550 — Introduction to Computer Graphics

Project 1: Bounding Spheres, Trackball, and Silhouettes

Student	Nguyen Khang Le	**OSU ID**	934768758
Course	CS 550	**Date**	October 9, 2026

Summary. This project implements automatic model normalization using a bounding sphere, quaternion-based trackball rotation, and silhouette rendering.

1. Bounding sphere

1(a). Computation and normalization

For the vertex set V, I compute the object-space center using the same unweighted-center formula as the provided definitions:

$$c_v = \frac{1}{|V|} \sum_{v \in V} v.$$

The radius is the greatest distance from this center to a vertex:

$$r_v = \max_{v \in V} \|v - c_v\|.$$

This is the **Unweighted Center** definition in the provided formulas document: every vertex in `Mesh::vertices` contributes equally to the center. In the implementation, V corresponds to `Mesh::vertices` and each v corresponds to `v->position`.

The model matrix applies the transformation

$$p' = \frac{p - c_v}{r_v}.$$

Thus the center is at the origin and every vertex is inside a sphere of radius one. This is implemented in `Mesh::compute_properties()` and applied before the camera view matrix in `display()`.

1(b). Alternative bounding sphere

The center used above is the **unweighted center**: every vertex contributes equally, regardless of the size of the surface region around it. This can be problematic for a poorly meshed model. For example, a small region with many vertices can pull the center toward that region even though it represents only a small part of the surface.

Following the second center definition in the provided formulas document, an alternative is the **area-weighted center**. For each triangle t, first compute its center:

$$C(t) = \frac{v_0 + v_1 + v_2}{3}.$$

Then weight each triangle center by its area:

$$c_A = \frac{\sum_{t \in T} A(t) C(t)}{\sum_{t \in T} A(t)},$$

where $A(t)$ is the area of triangle t. The radius is then recomputed using this center:

$$r_A = \max_{v \in V} \|v - c_A\|.$$

Unlike the unweighted center, this method gives larger surface regions more influence and reduces the effect of uneven vertex density. It can therefore place the center more appropriately and produce a smaller radius for an asymmetrically or poorly tessellated model, although it is not guaranteed to be smaller for every model. The tradeoff is that it requires using triangle areas and triangle centers instead of only looping over vertices.

The illustration should show the same asymmetric point set or mesh with the unweighted center and sphere beside the area-weighted center and sphere. Label the centers c_v and c_A, and label the radii r_v and r_A.

1(a) results

The required screenshots for Part 1(a), together with the screenshots for Parts 2(a) and 3(a), are collected in the appendix as one 2-by-2 grid per model. The suffixes are `00` for the original view, `01` for the bounding-sphere-adjusted view, `10` for silhouette rendering, and `11` for natural normal rendering.

2. Trackball interface

2(a). Trackball implementation and natural orientations

Mouse coordinates are converted to normalized coordinates and projected onto a virtual unit sphere. Points outside the disk are placed on the equator. For old and new projected points p_0 and p_1, the rotation axis and angle are

$$a = \frac{p_0 \times p_1}{\| p_0 \times p_1 \|}$$

$$\theta = \arccos(\text{clamp}(p_0 \cdot p_1, -1, 1)).$$

The quaternion `angleAxis(theta, a)` is multiplied into the camera orientation. The lower-right panel of each model grid shows its selected natural orientation using normal wireframe rendering.

2(b)(ii). Tetrahedron rotational symmetry group

The four specified tetrahedron vertices are permuted by every rotational symmetry. The rotation group is the even permutation group A_4, so it has $|A_4| = 12$ elements. They consist of the identity, eight rotations of 120 or 240 degrees about vertex-to-opposite-face axes, and three 180-degree rotations about opposite-edge axes.

Quaternion convention: $q = (w, x, y, z)$, with the vector part equal to $\sin(\frac{\theta}{2})a$ for a unit axis a.

Quaternion	**Axis**	**Angle**	**Matrix**
$(1, 0, 0, 0)$	–	0°	$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$
$\left(\frac{1}{2}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2}\right)$	$(1, 1, 1)$	120°	$\begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$
$\left(-\frac{1}{2}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2}\right)$	$(1, 1, 1)$	240°	$\begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$
$\left(\frac{1}{2}, \frac{1}{2}, -\frac{1}{2}, -\frac{1}{2}\right)$	$(1, -1, -1)$	120°	$\begin{pmatrix} 0 & 0 & -1 \\ -1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$
$\left(-\frac{1}{2}, \frac{1}{2}, -\frac{1}{2}, -\frac{1}{2}\right)$	$(1, -1, -1)$	240°	$\begin{pmatrix} 0 & -1 & 0 \\ 0 & 0 & 1 \\ -1 & 0 & 0 \end{pmatrix}$
$\left(\frac{1}{2}, -\frac{1}{2}, \frac{1}{2}, -\frac{1}{2}\right)$	$(-1, 1, -1)$	120°	$\begin{pmatrix} 0 & 0 & 1 \\ -1 & 0 & 0 \\ 0 & -1 & 0 \end{pmatrix}$

Quaternion	Axis	Angle	Matrix
$\left(-\frac{1}{2}, -\frac{1}{2}, \frac{1}{2}, -\frac{1}{2}\right)$	$(-1, 1, -1)$	240°	$\begin{pmatrix} 0 & -1 & 0 \\ 0 & 0 & -1 \\ 1 & 0 & 0 \end{pmatrix}$
$\left(\frac{1}{2}, -\frac{1}{2}, -\frac{1}{2}, \frac{1}{2}\right)$	$(-1, -1, 1)$	120°	$\begin{pmatrix} 0 & 0 & -1 \\ 1 & 0 & 0 \\ 0 & -1 & 0 \end{pmatrix}$
$\left(-\frac{1}{2}, -\frac{1}{2}, -\frac{1}{2}, \frac{1}{2}\right)$	$(-1, -1, 1)$	240°	$\begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & -1 \\ -1 & 0 & 0 \end{pmatrix}$
$(0, 1, 0, 0)$	$(1, 0, 0)$	180°	$\begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \end{pmatrix}$
$(0, 0, 1, 0)$	$(0, 1, 0)$	180°	$\begin{pmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & -1 \end{pmatrix}$
$(0, 0, 0, 1)$	$(0, 0, 1)$	180°	$\begin{pmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$

The list is complete because the identity contributes one element, each of the four vertices gives two nonidentity 3-fold rotations, and each of the three pairs of opposite edges gives one half-turn. Their totals are $1 + 4(2) + 3 = 12$, matching $|A_4|$.

3. Silhouette rendering

3(a). Method and results

The program first renders the triangles as a white depth-tested surface. It then transforms each triangle normal using the inverse-transpose normal matrix. For an edge with adjacent faces having view-space facing values $d_i = n_i \cdot (0, 0, 1)$, the edge is drawn when

$$d_0 d_1 < 0.$$

Boundary edges are also drawn. Replace the lower-left panel of every model grid with the corresponding silhouette result.

3(b). Discussion and improvements

The technique clearly emphasizes the outer contour, but line quality can be affected by mesh tessellation, aliasing, and inconsistent normals. Two potential improvements are:

1. Use anti-aliased, view-dependent line width so thin and thick regions remain visually consistent.
2. Improve treatment of boundary edges and sharp creases, possibly by using a crease-angle threshold and a dedicated outline pass.

Insert close-up images here showing the specific artifacts observed in the rendered models.

Conclusion

The completed viewer normalizes arbitrary meshes, supports interactive quaternion trackball rotation, and extracts silhouette edges from adjacent face orientations. The tetrahedron analysis independently confirms the twelve rotational symmetries required for the CS 550 portion of the assignment.

Appendix: Combined screenshot grids

For each model, the existing `00` and `01` figures form the top row. Add the Part 3(a) silhouette screenshot as `10` and the Part 2(a) natural normal screenshot as `11`. The final order is therefore: original, bounding-sphere adjusted, silhouette, and natural normal rendering.

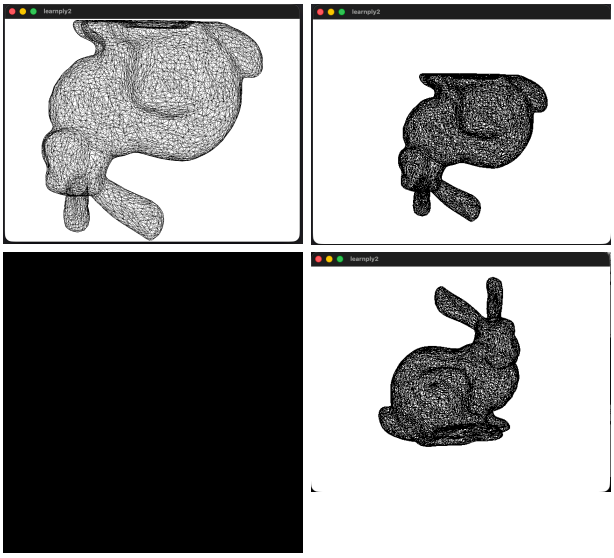


Figure 1: **bunny + ".ply"**

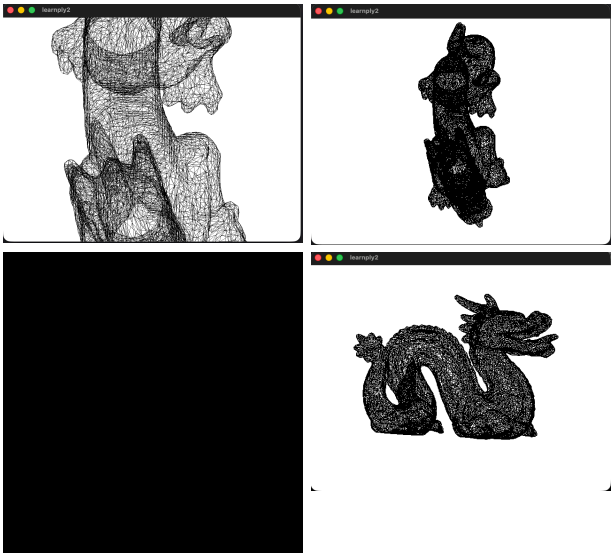


Figure 2: **dragon + ".ply"**

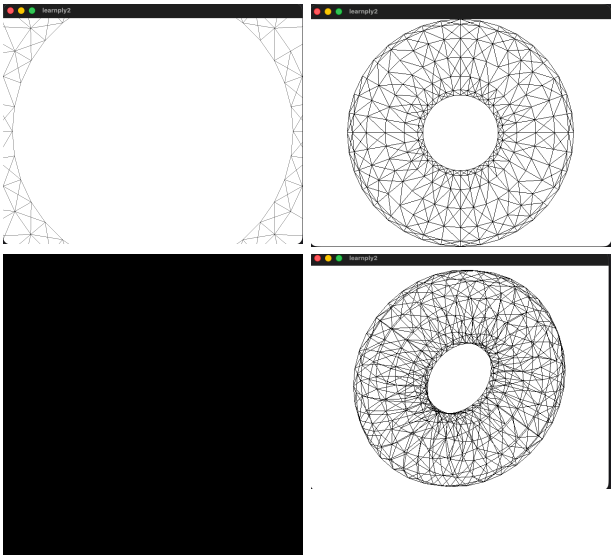


Figure 3: **torus + ".ply"**

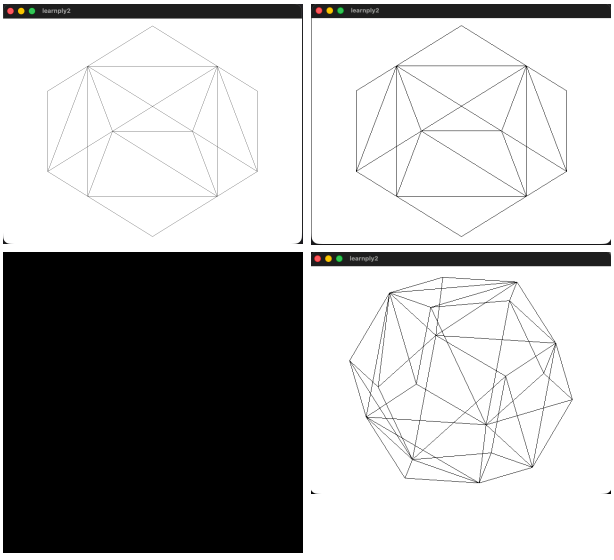


Figure 4: **dodecahedron + ".ply"**

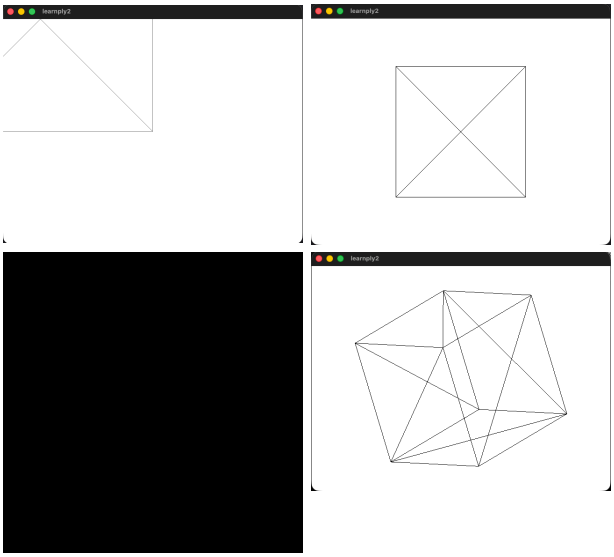


Figure 5: **cube + ".ply"**

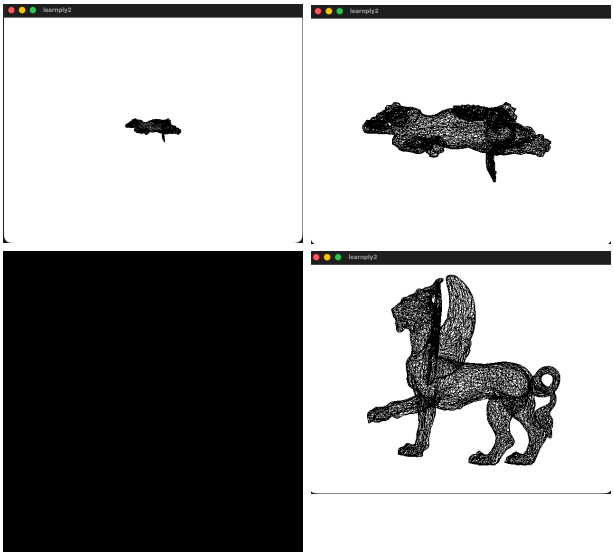


Figure 6: **feline + ".ply"**

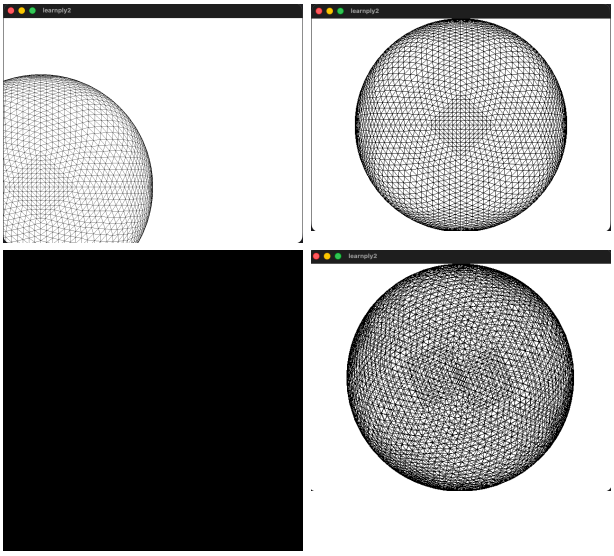


Figure 7: **sphere + ".ply"**

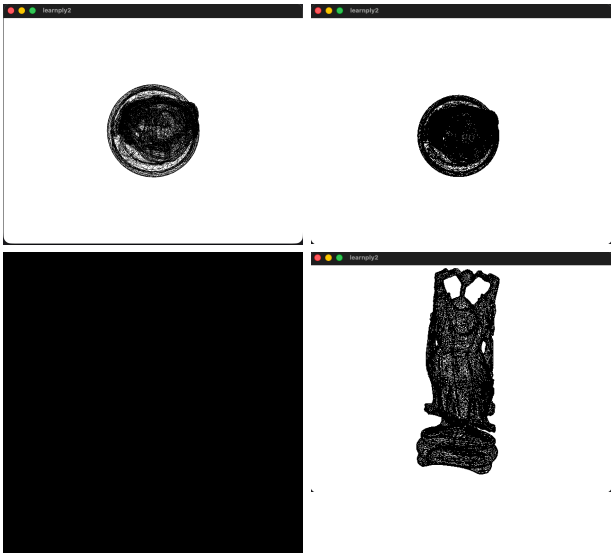


Figure 8: **happy + ".ply"**

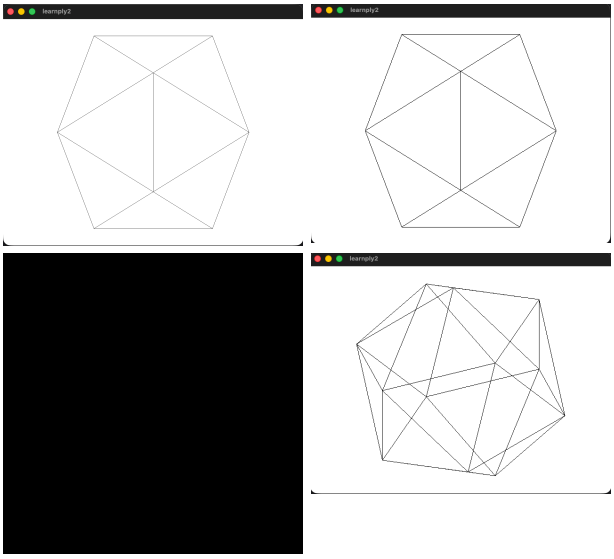


Figure 9: **icosahedron + ".ply"**

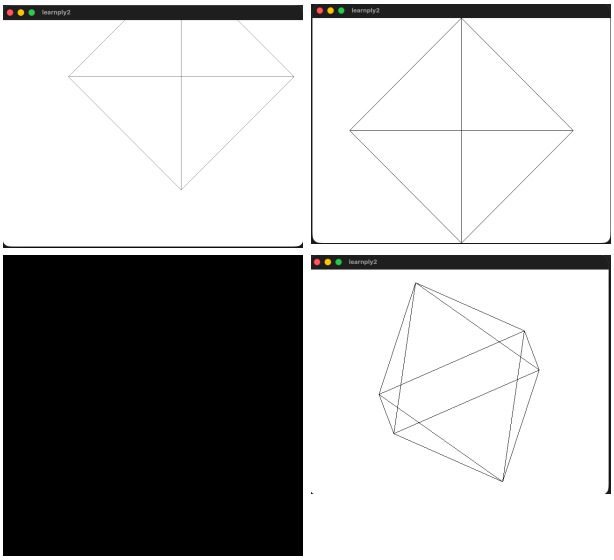


Figure 10: **octahedron + ".ply"**

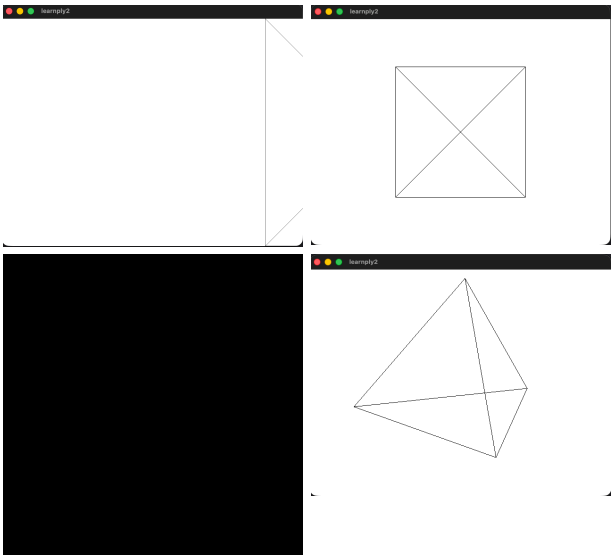


Figure 11: **tetrahedron + ".ply"**

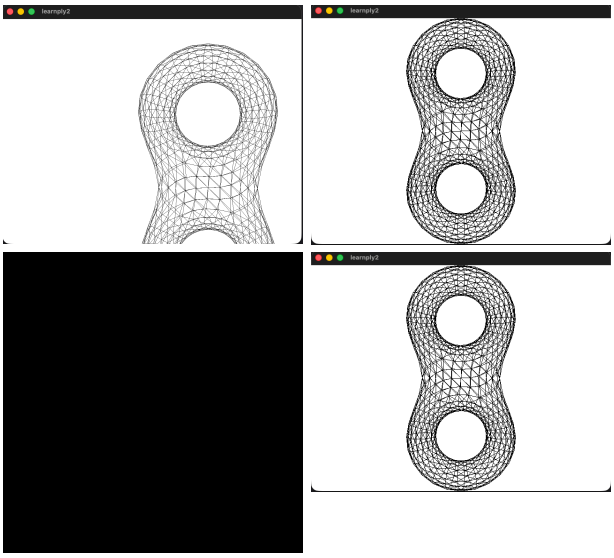


Figure 12: **eight + ".ply"**